



ESCOLA SUPERIOR **NÁUTICA** INFANTE D. HENRIQUE

Relatório do Projeto

Computação Distribuída

Projeto Supermercado XPTO

Rodrigo Ribeiro 14301

Fernando Simões 14041

Professor: Carlos Gonçalves

Ano Letivo 2025/2026

Conteúdo

Introdução	2
1 Repositório e Instruções de Instalação	3
Instruções para Recriar o Ambiente	3
2 Fase 1 - 21/04/2026	4
Objetivo 1: Separar a persistência de dados do backend Web	4
Objetivo 2: Comunicação por mensagens JSON	4
Objetivo 3: Contentorização com Docker Compose	4
Objetivo 4: Só o contentor Web é acessível do exterior	4
Resultados	4
Arquitetura	4
3 Fase 2 – 17/05/2026	5
Objetivo 1: Introdução de uma API REST como intermediário	5
Objetivo 2: Manutenção da comunicação TCP entre a API e a Base de Dados	5
Objetivo 3: Arquitetura com três contentores	5
Objetivo 4: Isolamento da Base de Dados	5
Objetivo 5: Único ponto de acesso exterior	5
UPDATE 11/06/2026 – Refatorização com Operações CRUD	6
Resultados	6
Arquitetura	6
3.0.1 Excerto de Código – Docker Compose com Redes Isoladas	7
3.0.2 Operações CRUD – Protocolo de Comunicação Refatorizado	8
4 Fase 3 – 12/06/2026	9
Abordagem escolhida: Integração no Backend	9
Objetivo 1: Acesso via API REST	9
Objetivo 2: Acesso via MQTT	9
Objetivo 3: Implementação	9
Dificuldades	9
UPDATE 15/06/2026 – Credenciais MQTT e atualização da API de callbacks	10
Resultados	10
Comparação entre REST Polling e MQTT Push	10
5 Utilização de Inteligência Artificial	12
6 Conclusão	13

Introdução

Este relatório documenta o desenvolvimento do projeto prático da cadeira de Computação Distribuída, que consistiu em transformar progressivamente uma aplicação web monolítica de um supermercado online ("Supermercado XPTO"), originalmente desenvolvida na cadeira de Programação Web, numa arquitetura distribuída baseada em contentores Docker.

O projeto foi desenvolvido em três fases incrementais, cada uma acrescentando uma nova camada de complexidade arquitetural: separação da persistência de dados, introdução de uma API REST intermédia, e integração de dados IoT em tempo real.

1 Repositório e Instruções de Instalação

O código fonte do projeto está disponível em:

https://github.com/14041-enidh/14041_14301_CD/tree/main/Projeto

Sendo a Fase 3, e última versão:

https://github.com/14041-enidh/14041_14301_CD/tree/main/Projeto/14041_14301_CD_3

Instruções para Recriar o Ambiente

Transferir o projeto

Aceder ao repositório no GitHub e selecionar Code -> Download ZIP. Após a transferência, extrair o conteúdo do ficheiro ZIP para uma pasta à escolha.

Aceder à pasta da aplicação

Abrir um terminal na pasta app do projeto extraído.

Construir e iniciar todos os contentores

```
docker compose up --build
```

Aceder à aplicação no browser

<http://localhost>

Parar os contentores

fazer ^C

Credenciais de teste

Email	Password	Perfil
teste@gmail.com	aaa	Utilizador normal
admin@gmail.com	admin	Administrador

2 Fase 1 - 21/04/2026

Nesta primeira fase do projeto de **Computação Distribuída** foram-nos pedidos diferentes objetivos, os quais com a aprendizagem dos laboratórios, fomos capazes de cumprir. A ideia principal desta fase era adaptar o trabalho prático desenvolvido na Unidade Curricular de Programação Web para um ambiente de contentores de acordo com os seguintes objetivos:

Objetivo 1: Separar a persistência de dados do backend Web

Antes, o nosso `Server.py` lia e escrevia ficheiros JSON diretamente no disco com `open()`. Agora essa responsabilidade passou para um servidor à parte, `db`. As funções `loadData` e `saveData` deixaram de usar `open()` para ler/escrever ficheiros diretamente no disco. Passaram a abrir uma ligação **TCP** ao servidor `db` na porta 12345. Isto separa fisicamente a lógica de negócio da lógica de armazenamento, que era o requisito central da fase.

Objetivo 2: Comunicação por mensagens JSON

As mensagens trocadas via socket seguem um protocolo simples com um campo `action` (`load` ou `save`), o nome do ficheiro, e no caso do `save`, os dados. **JSON** foi a escolha natural por ser legível, fácil de serializar em Python, e por ser o formato já usado em toda a aplicação.

Objetivo 3: Contendorização com Docker Compose

O `compose.yml` define uma rede **bridge** privada (`app_network`, subnet `192.168.100.0/24`) e os dois serviços dentro dela. O **Docker** resolve automaticamente o nome `db` como `hostname`, o que é por isso que no `Server.py` basta escrever `host = 'db'` em vez de um IP fixo.

Objetivo 4: Só o contentor Web é acessível do exterior

No `compose.yml`, apenas o `flask_app` tem `ports: "80:80"`, enquanto o contentor `db` não tem nenhum mapeamento de portas, o que significa que a porta 12345 nunca é acessível desde a web, ficando exclusivamente acessível dentro da rede **Docker** interna.

Resultados

O `Server.py` deixou de aceder ao disco diretamente, todas as leituras e escritas passam pelo socket **TCP**.

A porta 12345 do contentor `db` não está mapeada para o exterior, sendo acessível apenas dentro da rede **Docker**.

A aplicação web funciona normalmente: login, registo, carrinho de compras, confirmação de email.

Arquitetura

[Browser] → [flask_app :80] → (TCP :12345) → [db]

3 Fase 2 – 17/05/2026

Nesta segunda fase do projeto de **Computação Distribuída**, o objetivo principal foi introduzir uma **API REST** como intermediário entre o backend Web e a base de dados, tornando a arquitetura mais modular, segura e alinhada com os princípios de sistemas distribuídos.

Objetivo 1: Introdução de uma API REST como intermediário

Na Fase 1, o `Server.py` comunicava diretamente com o servidor de base de dados via socket **TCP**. Na Fase 2, essa comunicação direta foi eliminada. Foi criado um novo servidor intermédio, o `Server_api.py`, que funciona como ponte entre os dois, usando métodos **HTTP**, `GET /load` para carregar dados e `POST /save` para os guardar. O `Server.py` passou a enviar pedidos HTTP a este servidor para carregar ou guardar dados, sem precisar de saber como esses dados são guardados.

Objetivo 2: Manutenção da comunicação TCP entre a API e a Base de Dados

O `Server_db.py` manteve-se sem alterações. A **API REST** é que assume agora o papel de cliente TCP, comunicando com o servidor de base de dados na porta 12345 através de mensagens **JSON** com o campo `action` (`load` ou `save`). Esta abordagem reutilizou o protocolo já implementado na Fase 1, minimizando o trabalho necessário.

Objetivo 3: Arquitetura com três contentores

O `compose.yml` foi atualizado para definir três serviços: `flask_app`, `api` e `db`. Foram criadas duas redes **Docker** distintas: a `frontend_network` (10.10.1.0/24) que liga o `flask_app` à `api`, e a `backend_network` (10.10.2.0/24) que liga a `api` ao `db`. Foi também criado um `DockerfileAPI` para construir a imagem do contentor da API, utilizando a porta 5000.

Observação: A subnet foi alterada de 192.168.100.0/24 para 10.10.1.0/24 e 10.10.2.0/24 porque a rede criada na Fase 1 já ocupava o intervalo anterior, e o Docker não permite duas redes com endereços sobrepostos.

Objetivo 4: Isolamento da Base de Dados

O contentor `db` está exclusivamente na `backend_network`, sendo completamente inacessível a partir do exterior e também inacessível pelo `flask_app` diretamente. Apenas o contentor `api` consegue comunicar com ele. Isto garante um isolamento mais forte do que na Fase 1, onde o `flask_app` comunicava diretamente com o `db`.

Objetivo 5: Único ponto de acesso exterior

Apenas o contentor `flask_app` tem mapeamento de portas (80:80), sendo o único acessível pelo browser do utilizador. A `api` e o `db` não têm portas expostas ao exterior, garantindo que toda a comunicação passa obrigatoriamente pelo backend Web.

UPDATE 11/06/2026 – Refatorização com Operações CRUD

Após revisão do professor, foi identificada uma ineficiência na implementação anterior: sempre que era necessário aceder a um dado específico (por exemplo, um utilizador), o sistema carregava o ficheiro JSON completo e filtrava em Python o registo pretendido.

Para corrigir isto, foi refatorizado o protocolo de comunicação entre o `Server.py` e o `Server_db.py`, substituindo as ações genéricas `load` e `save` por operações **CRUD** direcionadas. Foram implementadas as seguintes ações:

- `get_user` (obtém um utilizador pelo email);
- `save_user` (atualiza ou cria um utilizador);
- `get_all_products` (devolve todos os produtos);
- `get_product_by_name` (obtém um produto pelo nome);
- `get_confirmation`, `save_confirmation` e `delete_confirmation` (gestão de tokens de confirmação de email).

Resultados

O db tornou-se completamente inacessível ao `flask_app` apenas o `api` consegue comunicar com ele.

As operações **CRUD** reduziram a quantidade de dados transferidos em cada pedido, evitando carregamentos desnecessários de informação.

A separação consiste agora em três camadas (Web / API / DB).

Arquitetura

```
[Browser] → [flask_app :80] → (HTTP :5000) → [api] → (TCP :12345) → [db]
                frontend_network                backend_network
                (10.10.1.0/24)                (10.10.2.0/24)
```

3.0.1 Excerto de Código – Docker Compose com Redes Isoladas

O Listing 1 apresenta o ficheiro `compose.yml` da Fase 2, onde se destacam a definição das duas redes isoladas e a forma como cada contentor é associado apenas às redes de que necessita.

Listing 1: `compose.yml` – arquitetura com três contentores e duas redes

```
1 networks:
2   frontend_network:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 10.10.1.0/24
7
8   backend_network:
9     driver: bridge
10    ipam:
11      config:
12        - subnet: 10.10.2.0/24
13
14 services:
15   flask_app:
16     container_name: flask_app_container
17     build:
18       context: .
19       dockerfile: Dockerfile
20     ports:
21       - "80:80"
22     networks:
23       - frontend_network
24     depends_on:
25       - api
26
27   api:
28     container_name: api_container
29     build:
30       context: .
31       dockerfile: DockerfileAPI
32     networks:
33       - frontend_network
34       - backend_network
35     depends_on:
36       - db
37
38   db:
39     container_name: db_container
40     build:
41       context: .
42       dockerfile: DockerfileDB
43     networks:
44       - backend_network
```


O ponto-chave desta configuração é que o contentor `flask_app` pertence exclusivamente à `frontend_network`, e o contentor `db` pertence exclusivamente à `backend_network`. Apenas o contentor `api` está presente em ambas as redes, funcionando como ponte controlada entre as duas camadas. Isto significa que, ao nível de rede, é fisicamente impossível o `flask_app` comunicar diretamente com o `db`, mesmo que o código tentasse fazê-lo.

3.0.2 Operações CRUD – Protocolo de Comunicação Refatorizado

A Tabela 1 descreve as sete operações implementadas após a refatorização solicitada pelo docente, substituindo as ações genéricas `load` e `save` por operações direcionadas a registos específicos.

Tabela 1: Operações CRUD implementadas no protocolo TCP/JSON

Operação	Parâmetros	Descrição
<code>get_user</code>	<code>email</code>	Devolve o utilizador com o email indicado
<code>save_user</code>	<code>email, data</code>	Atualiza ou cria um utilizador
<code>get_all_products</code>	–	Devolve a lista completa de produtos
<code>get_product_by_name</code>	<code>name</code>	Devolve o produto com o nome indicado
<code>get_confirmation</code>	<code>id</code>	Obtém o email associado a um token de confirmação
<code>save_confirmation</code>	<code>id, email</code>	Guarda um novo token de confirmação de email
<code>delete_confirmation</code>	<code>id</code>	Remove um token após ser utilizado

A principal vantagem desta abordagem face às ações genéricas anteriores é a eficiência: antes, obter um único utilizador implicava transferir via socket o ficheiro

4 Fase 3 – 12/06/2026

Nesta terceira fase do projeto, o objetivo foi integrar dados de sensores **IoT** (Temperatura e Humidade) provenientes do laboratório do docente na interface da aplicação web do Supermercado XPTO, através de duas formas distintas de comunicação: **API REST** e **MQTT**.

Abordagem escolhida: Integração no Backend

A integração foi implementada no servidor Flask (`Server.py`), sem alterações aos contentores Docker nem à estrutura da aplicação. Esta abordagem evita expor endereços externos no código do browser, resolve automaticamente as restrições de **CORS** e permite utilizar bibliotecas Python nativas para comunicação com os sistemas IoT.

Objetivo 1: Acesso via API REST

Foi implementada a rota `/weather/rest` no `Server.py`, que usa a biblioteca `requests` para interrogar o endpoint REST do docente (`https://cjsg.ddns.net:8443/weather/values`) a cada vez que é chamada. O frontend faz `fetch('/weather/rest')` ao próprio servidor a cada 5 segundos, atualizando automaticamente os valores de temperatura e humidade no dashboard.

Objetivo 2: Acesso via MQTT

Foi instalada a biblioteca `paho-mqtt` no contentor Flask. Quando o servidor arranca, é criada automaticamente uma ligação ao servidor do docente (`cjsg.ddns.net`) na porta 1883, que fica em segundo plano à espera de mensagens do tópico `weather`. Sempre que o sensor publica dados, estes são guardados em memória e disponibilizados através da rota `/weather/mqtt`. Comparando com a **REST API**, este modelo é mais adequado para dados em tempo real pois não requer pedidos constantes ao servidor externo.

Objetivo 3: Implementação

Foi adicionada uma secção visual tanto na página de login (`index.html`) como na página principal (`HomePage.html`), com dois blocos distintos, um para os dados **REST** e outro para os dados **MQTT**, atualizados de 5 em 5 segundos via `fetch` ao backend.

Dificuldades

Durante os testes, o endpoint REST devolveu dados corretamente, no entanto, o cliente MQTT conectou-se com sucesso ao broker mas permaneceu em estado de espera (**A aguardar...**), o que reflete um erro da parte do servidor do docente, e não um erro no código implementado.

UPDATE 15/06/2026 – Credenciais MQTT e atualização da API de callbacks

Após indicação do docente, foram adicionadas as credenciais de acesso ao servidor MQTT (utilizador: `cd`, password: `1qaz"WSX`), que eram necessárias para estabelecer a ligação autenticada. Foi também atualizada a inicialização do cliente MQTT para usar `CallbackAPIVersion.VERSION2`, eliminando o aviso de "Callback API version 1 is deprecated, update to latest version" que aparecia nos logs do servidor.

No entanto, os dados MQTT continuam sem aparecer no dashboard, o que indica que o sensor do docente não está a publicar mensagens neste momento. Assim que voltar a publicar, os dados aparecerão automaticamente.

Resultados

A rota `/weather/rest` devolve dados corretamente.

A rota `/weather/mqtt` estabelece ligação com sucesso ao broker (confirmado nos logs), mas permanece em estado de espera devido à ausência de publicações do sensor do docente durante o período de testes.

O dashboard **IoT** é apresentado em `index.html` (na página de login)¹ e `HomePage.html` (Na página principal, ao lado do mapa)², com dois blocos distintos (REST e MQTT), atualizados automaticamente de 5 em 5 segundos.

Comparação entre REST Polling e MQTT Push

A Tabela 2 compara os dois métodos de integração IoT implementados, evidenciando as diferenças arquiteturais e as situações em que cada um é mais adequado.

Tabela 2: Comparação entre REST polling e MQTT push

Critério	REST polling	MQTT push
Modelo de comunicação	Pull – o cliente pede periodicamente	Push – o broker envia quando há dados
Ligações ao servidor externo	1 por cada intervalo de polling (5 s)	1 ligação persistente
Latência dos dados	Até 5 segundos	Imediata
Consumo de recursos	Maior (pedidos constantes)	Menor (ligação permanente leve)
Complexidade de implementação	Baixa	Média
Adequação a tempo real	Moderada	Alta
Resultado nos testes	Dados recebidos corretamente	Ligado ao broker; sensor inativo

Login

Email:

Senha:

Entrar

[Não tem conta? Crie uma!](#)

Painel IoT

REST: 26.37°C / 45.56%

MQTT: A aguardar...

Figura 1: index.html

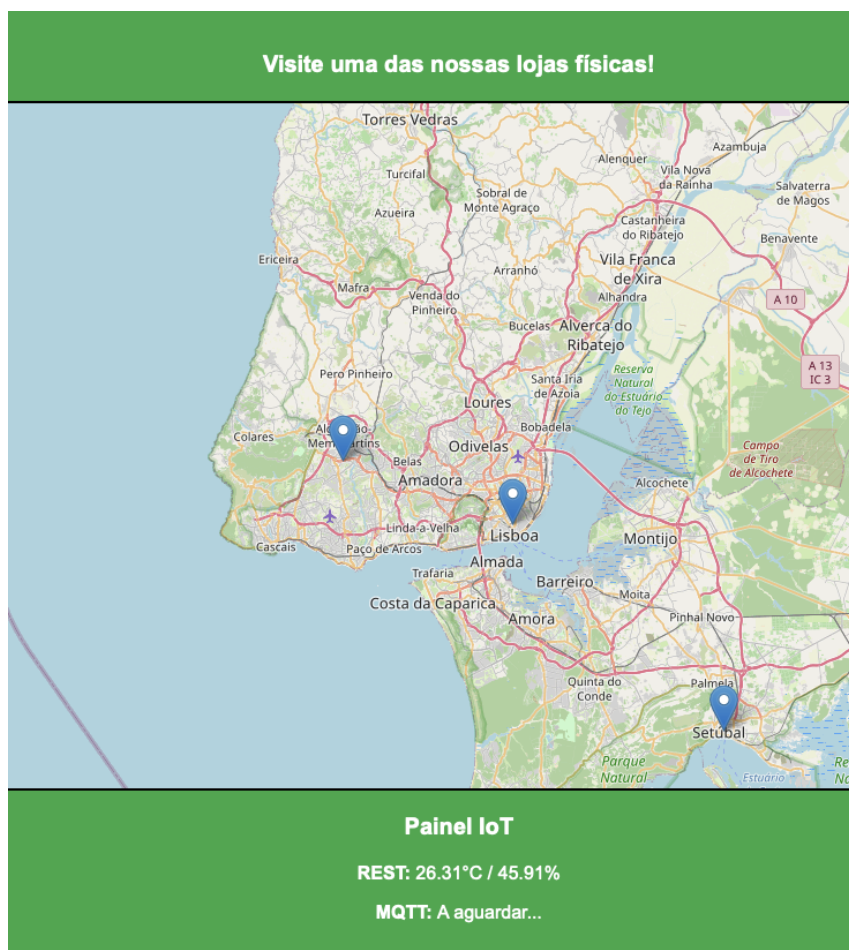


Figura 2: HomePage.html

5 Utilização de Inteligência Artificial

Ao longo do desenvolvimento deste projeto, foi utilizada a IA Claude (Anthropic) como ferramenta de apoio.

Como foi utilizado

Resolução de erros: Diagnóstico e correção de falhas de socket (deadlocks no ciclo de `recv`), problemas de ligação entre contentores Docker e avisos de depreciação na biblioteca `paho-mqtt`.

Revisão de código: Otimização do ficheiro `Server_db.py`, aplicando `os.path.basename()` para garantir a segurança dos caminhos de ficheiros e reorganizando a estrutura para operações **CRUD**.

Análise conceptual: Estudo sobre o funcionamento de redes Docker (DNS interno e isolamento de sub-redes), comparação entre **REST polling** e **MQTT push**, e configuração de `flask-session`.

O que não foi gerado por IA

Todo o código funcional foi escrito e testado pelos alunos. A IA foi utilizada como ferramenta de consulta e revisão, não como geradora autónoma de código submetido diretamente.

6 Conclusão

O desenvolvimento deste projeto ao longo das três fases permitiu construir progressivamente uma aplicação web distribuída, partindo de uma arquitetura simples, até uma solução mais complexa, mas organizada e melhor isolada, terminando com a implementação de **IoT**.

Na Fase 1 foi estabelecida a separação entre o backend Web e a persistência de dados através de comunicação **TCP**. Na Fase 2 essa arquitetura foi aprofundada com a introdução de uma **API REST** como intermediário, reforçando o isolamento entre camadas e aplicando posteriormente operações **CRUD** direcionadas para tornar a comunicação mais eficiente. Na Fase 3 a aplicação foi estendida ao mundo IoT, integrando dados reais de sensores através de dois métodos distintos, **REST** e **MQTT**.

Ao longo das três fases é possível identificar a presença dos cinco princípios fundamentais dos sistemas distribuídos.

- A separação em contentores independentes contribui para a **Resiliência**, uma vez que cada componente pode falhar ou ser reiniciado sem comprometer os restantes.
- A substituição das operações genéricas de leitura por operações **CRUD** direcionadas melhora o **Desempenho** e a **Eficiência**, evitando o carregamento desnecessário de dados.
- O modelo de redes isoladas, onde o contentor **db** só é acessível pela API, e a API só é acessível pelo Flask, implementa o princípio de **Zero-Trust**, em que nenhum componente confia diretamente noutro sem passar pela camada adequada.
- Por fim, a arquitetura modular baseada em contentores facilita a **Escalabilidade**, permitindo replicar ou substituir qualquer camada de forma independente.

O resultado final é uma aplicação funcional que demonstra na prática os principais conceitos de **Computação Distribuída**: comunicação entre processos, modelo cliente/servidor, contentorização, **Web Services REST** e protocolos **IoT**. Cada fase acrescentou uma camada de complexidade controlada, refletindo a forma como sistemas distribuídos reais evoluem, de forma incremental e modular.